Practical - 1

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

A. Theory & Analysis

1. What is URL and parts of URL?

A URL (Uniform Resource Locator) is the global identifier used to locate web resources across the Internet. Its primary components include: (1) Protocol/Scheme: Specifies the communication mechanism, such as HTTP or HTTPS; (2) Subdomain: Directs requests to specific service clusters (e.g., student.charusat.edu.in); (3) Domain Name: The registered host name of the university or organization; (4) Port Number: The network communication port (e.g., :8000 for local PHP development, :80 for HTTP, :443 for HTTPS); (5) Path: Identifies the exact hierarchical location of the resource on the server (e.g., /pages/dashboard.html); (6) Query String: Key-value parameters passed to server-side scripts beginning with '?' (e.g., ?role=student&status=active); and (7) Fragment Identifier: Anchors starting with '#' that instruct the browser to scroll directly to a specific HTML element ID (e.g., #faqTitle).

2. How does an HTML file get processed in a web browser?

When a browser receives an HTML document, it executes a sequential multi-stage rendering pipeline: (1) Byte Conversion & Decoding: Raw network byte streams are decoded into human-readable characters based on the declared character encoding (UTF-8); (2) Tokenization: The HTML parser breaks characters into distinct lexical tokens such as start tags, end tags, attributes, and text nodes; (3) DOM Tree Construction: Tokens are assembled into a hierarchical tree structure of node objects called the Document Object Model (DOM); (4) CSSOM Construction: External and embedded stylesheets are fetched and parsed into the CSS Object Model (CSSOM); (5) Render Tree Generation: The DOM and CSSOM are combined, filtering out non-rendered elements (like <head> or display:none) to determine computed styles; (6) Layout/Reflow: The browser calculates exact pixel coordinates and geometry for each element on the viewport; and (7) Painting & Compositing: Visual layers are painted onto screen pixels, delivering the interactive user interface.

3. How will page navigation flow be managed among all HTML pages?

Page navigation is structured through a standardized, persistent Bootstrap 5 navigation bar included across all pages. Consistent relative pathing (e.g., ../pages/, ../php/) is maintained to ensure seamless routing in local server environments. Public users can access informational pages (Home, About, Events, Contact, Login, and Registration). Upon successful authentication, role-based redirection routes student accounts to dashboard.php and administrator accounts to admin_dashboard.php. Direct access to protected internal modules enforces session verification middleware, ensuring unauthorized attempts redirect back to login.html with clear informative messages.

4. How will GitHub commits be maintained after each practical?

Git version control is maintained using an organized, milestone-driven repository workflow. Each practical is committed with clear semantic commit messages (e.g., 'feat: practical 1 requirement analysis and sitemap', 'feat: practical 8 normalized database schema and PDO connection'). A root .gitignore file excludes temporary system files (.DS_Store), cache artifacts, and user uploads. Practical milestones allow revision tracing, while the README.md provides comprehensive local installation instructions, database import procedures, and sample user credentials.

B. Implementation Details

The Student Hub Portal was initiated by defining the complete project scope for Charotar University of Science and Technology (CHARUSAT). A total of 16 functional pages were planned and created, including public portals, authenticated student modules (Dashboard, Attendance, Assignments, Courses, Timetable, Profile), and administrative modules (Admin Dashboard, Student CRUD, Event CRUD, Fetch API CRUD). The directory architecture separates concerns into api/, css/, data/, docs/, images/, js/, pages/, php/, sql/, storage/, and uploads/.

C. Evidence of Work

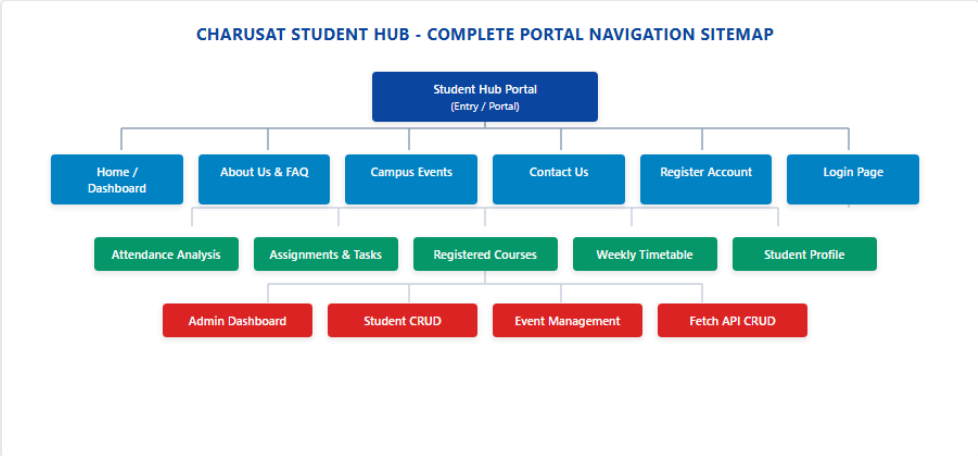


Figure: CHARUSAT Student Hub Complete Navigation Sitemap & Architecture

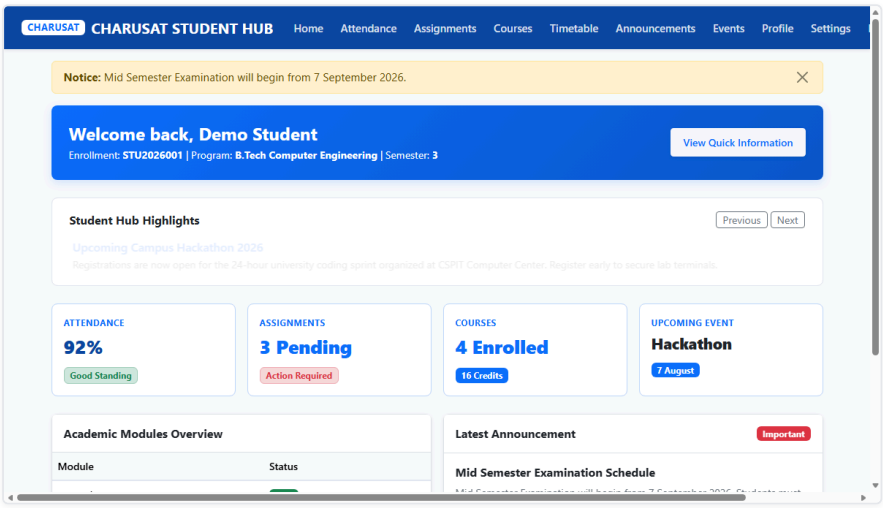


Figure: Student Hub Portal Primary Dashboard & Attendance KPI Overview

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to conduct structured software requirement analysis, design user navigation sitemaps, establish a maintainable project folder structure, and organize milestone-based version control for a multi-page university web application.

Faculty Remarks: (Keep this blank for faculty)

Practical - 2

Practical Title: Semantic HTML5 Pages with Accessibility-Ready Structure

Problem Definition: Develop static HTML5 skeletons for at least 10 StudentHub pages such as Home, About, Register, Login, Dashboard, Events, Profile, Contact, Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.

A. Theory & Analysis

1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?

Yes. Semantic HTML5 elements are employed across all portal pages to convey structural meaning to both web browsers and assistive screen readers. The <header> element encloses top branding and university credentials; <nav> encapsulates the main navigation links; <main> defines the core, unique content of each page; <section> organizes logical content blocks like attendance overviews and course tables; <article> wraps self-contained information units such as announcements and event cards; and <footer> encloses university copyright information and secondary navigation links. Using semantic elements eliminates excessive non-descriptive containers and greatly improves document outline clarity.

2. Are form labels, headings, image alt text, and navigation links accessible?

Yes, full accessibility compliance has been integrated. Every input field in the registration, login, and contact forms is explicitly associated with a <label for='inputId'> tag matching the input's unique ID. Headings follow a strict logical hierarchy from <h1> through <h5> without skipping levels. All images contain descriptive alternative text (e.g., alt='CHARUSAT Official Logo', alt='Student Profile Photo'), ensuring screen readers convey visual context. Interactive links feature legible text and high contrast ratios meeting WCAG 2.1 Level AA requirements.

3. Is the page structure consistent across all pages?

Yes. All pages share a standardized HTML template layout: strict HTML5 doctype declaration, UTF-8 character encoding, responsive viewport meta tags, sticky responsive Bootstrap 5 navbar, unified main content containers, and identical footer markup. This guarantees predictable visual rendering, seamless navigation, and straightforward code maintainability across all 16 pages.

B. Implementation Details

Semantic HTML5 skeletons were created and refined across 16 interconnected pages of the Student Hub Portal. Accessibility-ready structures were implemented in pages/about.html, pages/contact.html, pages/profile.html, pages/attendance.html, and pages/timetable.html with semantic markup, ARIA attributes, explicit form labeling, and descriptive image alt tags.

C. Evidence of Work

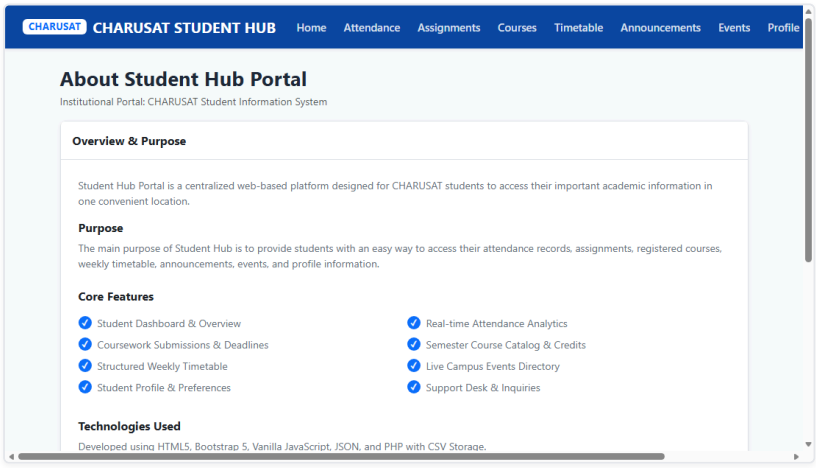


Figure: About Page Structure with Semantic Main, Section, and FAQ Accordion

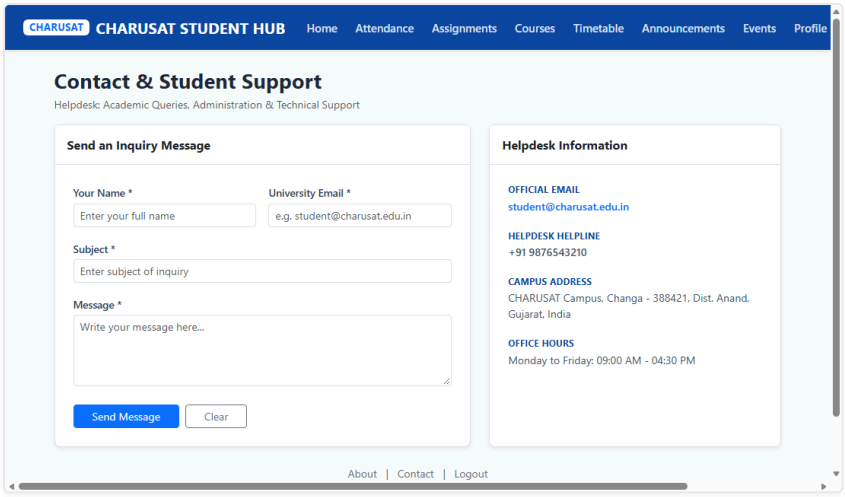


Figure: Accessible Contact Form with Explicit Label-Input Binding

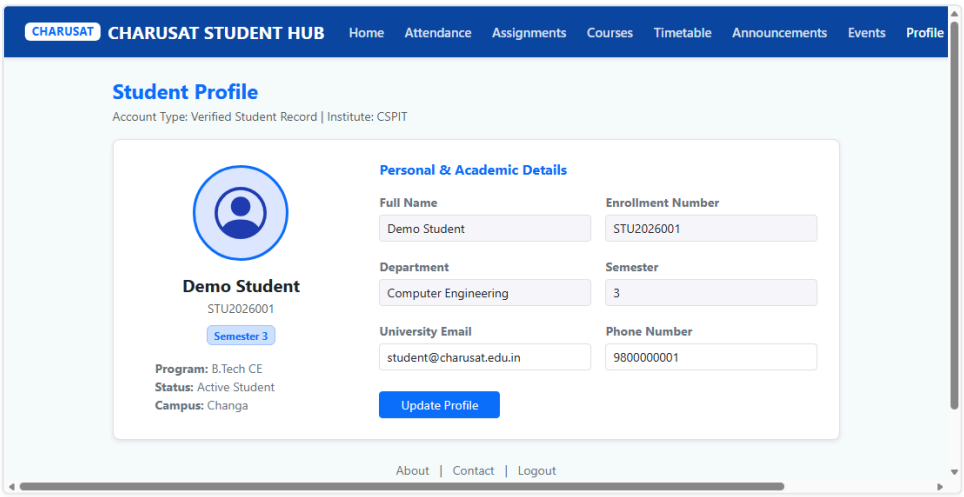


Figure: Student Profile Page with Heading Hierarchy and Form Attributes

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned to construct accessibility-ready web pages using semantic HTML5 elements, logical heading hierarchies, explicit form label associations, and descriptive image alternative text.

Faculty Remarks: (Keep this blank for faculty)

Practical - 3

Practical Title: Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

Problem Definition: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

A. Theory & Analysis

1. How does the layout adapt for mobile, tablet, and desktop screens?

The layout adopts a mobile-first responsive strategy powered by Bootstrap 5's responsive grid system and media queries. On mobile viewports (<576px and 768px), navigation links collapse into a hamburger menu, and multi-column grid layouts stack vertically into single columns (col-12) to prevent horizontal scrolling. On medium screens and tablets (768px - 991px), attendance cards and dashboard cards display in a balanced 2-column grid. On desktops (≥992px and 1200px), full multi-column layouts (col-lg-3, col-lg-4) expand to leverage widescreen displays with optimal whitespace and padding.

2. Which layout technique is used and why?

A hybrid layout technique combining CSS Flexbox and CSS Grid via Bootstrap 5 utilities is employed. Flexbox (d-flex, justify-content-between, align-items-center) is used for 1-dimensional component alignments, such as navbar brand items, card headers, and button groups. CSS Grid (row, g-3, col-*) is utilized for 2-dimensional page layouts, ensuring predictable alignment of attendance summary cards, event grids, and multi-column forms across varied viewports.

3. Are typography, spacing, colors, and contrast readable?

Yes. Clean system fonts (Segoe UI, Roboto, system-ui) are utilized with proportional font sizing and comfortable line heights. A cohesive university color scheme is enforced using primary blue (#0d6efd and #0d47a1) with high text contrast against white and light-gray backgrounds, achieving a contrast ratio well above the WCAG 4.5:1 standard. Spacing follows a consistent modular scale using Bootstrap's utility classes (m-*, p-*, g-*).

4. Is the framework used consistently without unnecessary inline CSS?

Yes. Bootstrap 5 classes are utilized systematically throughout the templates for layout, spacing, colors, and components. Custom styling is cleanly isolated within dedicated stylesheet files (e.g., css/dashboard.css, css/profile.css, css/events.css) rather than cluttered inline styles, preserving maintainability and separation of concerns.

B. Implementation Details

The responsive layout was implemented using Bootstrap 5.3.3 CDN, Flexbox utilities, and CSS Grid across all portal pages. The four attendance summary cards on the dashboard were unified using the university primary blue theme (#0d6efd) with identical border-radii, badges, and card elevations. Viewports at 1280px (desktop), 768px (tablet), and 375px (mobile) were thoroughly verified.

C. Evidence of Work

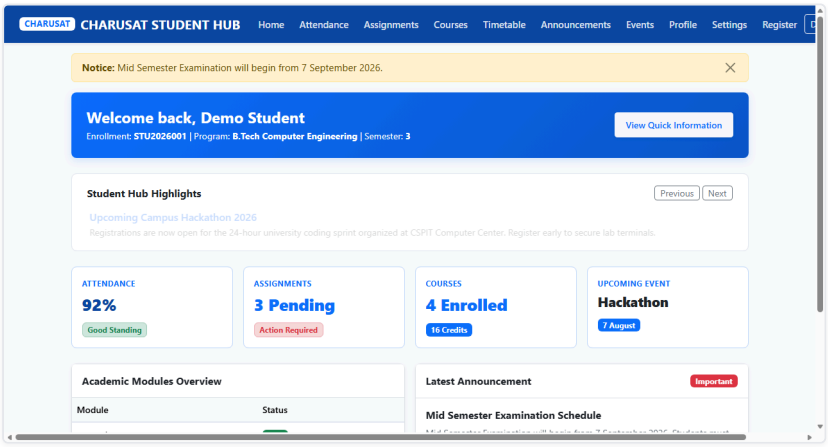


Figure: Desktop Layout (1280px Viewport) with 4-Column Grid Alignment

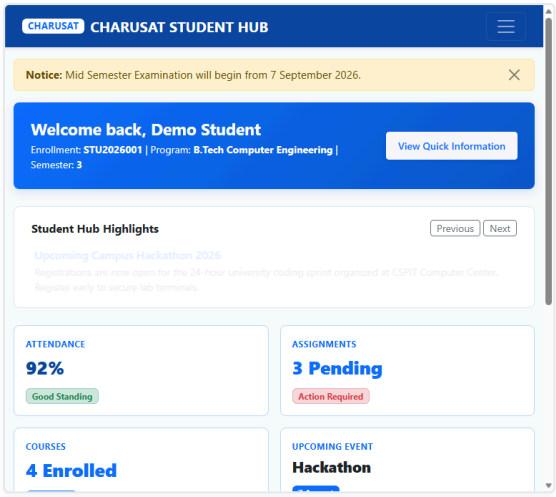


Figure: Tablet Layout (768px Viewport) with 2-Column Responsive Stacking

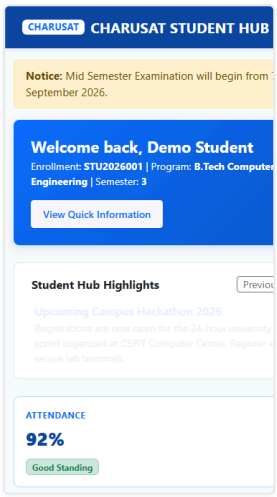


Figure: Mobile Layout (375px Viewport) with Fluid Single-Column Flow

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to implement mobile-first responsive web design using Bootstrap 5 grid systems, Flexbox alignment utilities, and media query breakpoints to deliver fluid user experiences across all screen sizes.

Faculty Remarks: (Keep this blank for faculty)

Practical - 4

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript DOM manipulation and localStorage.

A. Theory & Analysis

1. How are DOM elements selected and modified?

DOM elements are selected using standard JavaScript methods including document.getElementById() for unique IDs and document.querySelector() / querySelectorAll() for CSS selectors. Elements are modified dynamically by manipulating their class lists (classList.add(), classList.remove(), classList.toggle()), setting element content via textContent or innerHTML, and adjusting DOM attributes using setAttribute() and dataset properties.

2. Are event listeners attached correctly?

Yes. Event listeners are bound cleanly using the addEventListener() method for click and submit events. Code execution is deferred until the DOM is fully constructed using document.addEventListener('DOMContentLoaded', ...), ensuring elements are accessible before script execution and preventing null reference exceptions.

3. Is localStorage used for remembering theme choice?

Yes. When the student toggles between light and dark modes via the theme button, the active state is stored in localStorage.setItem('theme', 'dark'). On subsequent page loads or across different pages of the portal, a startup script retrieves localStorage.getItem('theme') and immediately applies the theme attribute (data-bs-theme='dark') to the document body, ensuring seamless persistence.

4. Does interactivity improve usability without breaking accessibility?

Yes. Interactive components enhance user engagement while strictly preserving accessibility. The collapsible FAQ updates aria-expanded attributes; the modal popup supports keyboard navigation and ESC-key dismissal; notification banners can be dismissed without shifting content awkwardly; and the theme switcher preserves high-contrast text ratios in both modes.

B. Implementation Details

Interactive JavaScript components were implemented across the portal: (1) Collapsible FAQ accordion in pages/about.html; (2) Quick Info modal popup in pages/dashboard.html; (3) Academic Highlights content slider in js/dashboard.js; (4) Dismissible notice banner; (5) Mobile hamburger menu; and (6) Global persistent light/dark theme switcher in js/common.js.

C. Evidence of Work

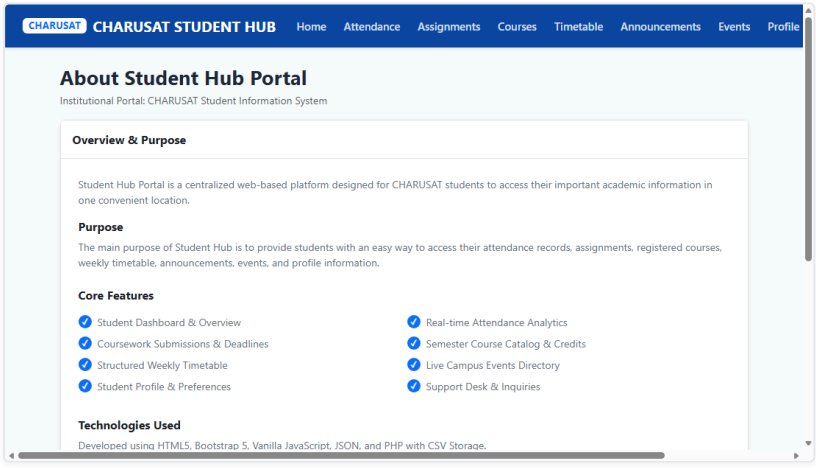


Figure: Interactive Collapsible FAQ Accordion with Dynamic Icon Toggling

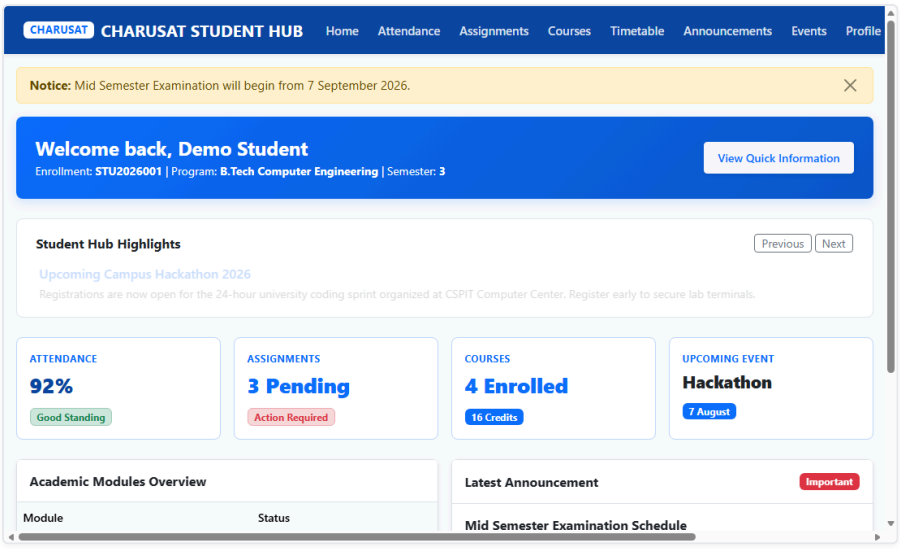


Figure: Dashboard with Interactive Carousel Slider and Dismissible Banner

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to manipulate the Document Object Model (DOM), handle user interaction events, build dynamic sliders and modals, and persist client preferences using browser localStorage.

Faculty Remarks: (Keep this blank for faculty)

Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory & Analysis

1. Are correct input types and attributes used?

Yes. The form utilizes specific HTML5 input types: type='text' for student name, type='email' for university email, type='tel' for mobile number, type='password' for secure password entry, <select> dropdowns for course and academic year, radio buttons for gender, and type='checkbox' for terms acceptance. Required attributes, pattern attributes, and autocomplete hints are configured appropriately.

2. Are validation errors displayed near the relevant fields?

Yes. Validation errors are presented inline directly below each offending input field using Bootstrap's invalid-feedback containers. When validation fails, the input element receives the is-invalid CSS class, turning the border red and revealing a clear, descriptive error message indicating how to correct the input.

3. Is password strength checked?

Yes. The password field listens to real-time input events and evaluates complexity against length, uppercase letters, lowercase letters, digits, and special characters. A visual strength meter dynamically updates color and width from Weak (red) to Medium (yellow) and Strong (green), guiding the student to choose a secure password.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Yes. Form fields feature explicit <label for='fieldId'> associations, aria-describedby links to help text and error messages, and a natural tab order allowing full keyboard-only navigation. Error messages are announced clearly by assistive screen-reading tools.

B. Implementation Details

A complete student registration form was developed in pages/register.html with validation logic in js/register.js. The validation enforces regular expressions: Name (/^[A-Za-z]{2,50}$/), University Email (/^[a-zA-Z0-9._%+-]+@charusat\.edu\.in$/i), Mobile Number (/^[6-9]\d{9}$/), and Password Complexity (/^(?=.*[A-Za-z])(?=.*\d).{6,}$/). Confirm password matching and terms verification are enforced before form submission.

C. Evidence of Work

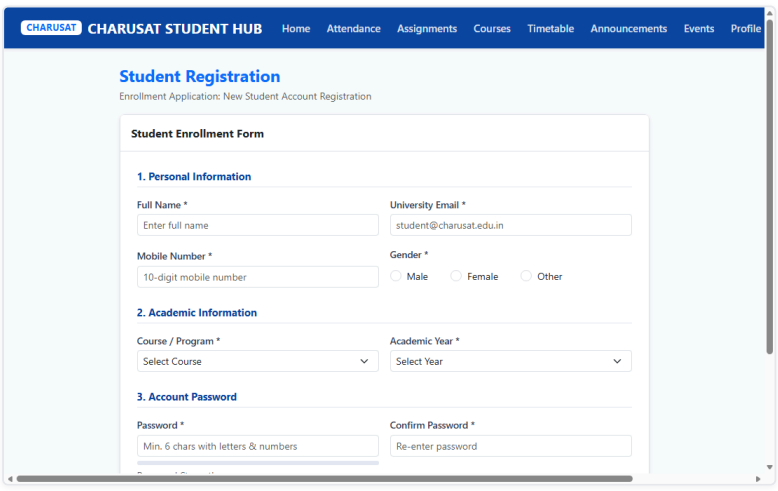


Figure: Student Registration Form with HTML5 Input Types and Validation Structure

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to build accessible forms and implement real-time client-side validation using regular expressions, password complexity meters, and user-friendly inline feedback.

Faculty Remarks: (Keep this blank for faculty)

Faculty Remarks: (Keep this blank for faculty)

Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic search, filter by category/course, sorting, pagination, and handle loading/error states.

A. Theory & Analysis

1. How is JSON fetched, parsed, and rendered?

Data is retrieved asynchronously using the standard Fetch API: fetch('../data/events.json'). The fetch returns a Promise resolved into an HTTP response object. The JSON payload is parsed using response.json(), and the resulting array of event objects is passed to a rendering function that dynamically creates Bootstrap event cards and appends them to the DOM container.

2. Which array methods are used for search, filter, sort, and pagination?

JavaScript array higher-order functions are used: (1) .filter() matches user query strings against event titles and venues, as well as category dropdowns; (2) .sort() arranges events chronologically by date or alphabetically; (3) .slice((page - 1) * perPage, page * perPage) divides the filtered array into paginated chunks; and (4) .map() / .forEach() constructs HTML template literals for rendering.

3. How are loading and error states handled?

Before the fetch initiates, a loading spinner ( ) is displayed. If network requests fail or return non-200 status codes, a prominent error alert ( ) informs the user. When a search or filter yields no matching results, a friendly 'No events found matching your criteria' empty-state card is rendered.

4. How is modularity maintained in JavaScript files?

Code is structured into dedicated modular functions within js/events.js: fetchEvents(), renderEvents(), applyFilterAndSearch(), setupPagination(), and updatePaginationControls(). This modular design keeps business logic cleanly separated from presentation.

B. Implementation Details

The events module in pages/events.html was integrated with an external JSON dataset (data/events.json) containing 15+ rich campus events. The script in js/events.js implements live search filtering, category filtering (Competition, Workshop, Seminar, Cultural, Sports), ascending/descending date sorting, and interactive pagination.

C. Evidence of Work

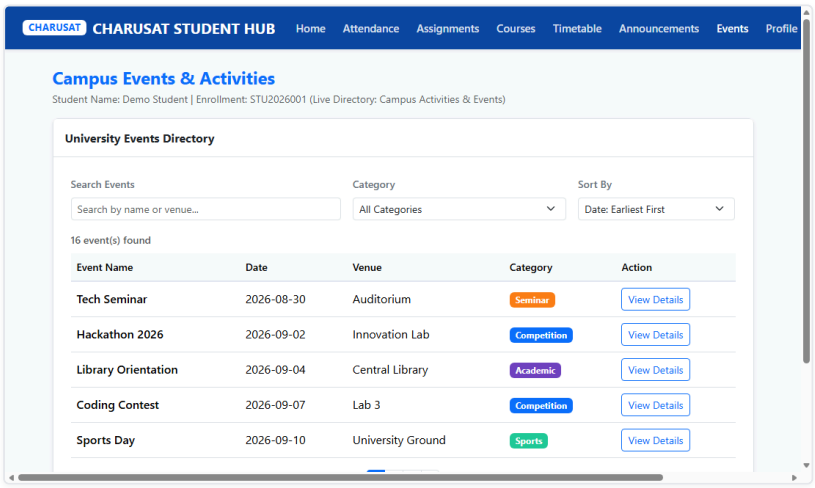


Figure: Campus Events Rendered Asynchronously from External JSON with Search and Filters

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to use the Fetch API to load and parse external JSON datasets asynchronously, and how to apply array operations to build client-side search, filtering, sorting, and pagination.

Faculty Remarks: (Keep this blank for faculty)

Faculty Remarks: (Keep this blank for faculty)

Practical - 7

Practical Title: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition: Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server, handle file writing safely into CSV/JSON format, and display clear success/error messages.

A. Theory & Analysis

1. Is the form submitted using POST?

Yes. Form submissions use the HTTP POST method (<form method='POST' action='../php/contact.php'>). The POST method packages data within the HTTP message body, preventing sensitive student information from appearing in URL query strings, server access logs, or browser histories.

2. Are inputs validated and sanitized server side?

Yes. The PHP script extracts fields via $_POST and applies rigorous validation: trim() strips extraneous whitespace, htmlspecialchars(..., ENT_QUOTES, 'UTF-8') neutralizes potential Cross-Site Scripting (XSS) characters, and filter_var($email, FILTER_VALIDATE_EMAIL) verifies RFC-compliant email formatting.

3. Is file writing handled safely?

Yes. File operations utilize safe appending modes (fopen($file, 'a')). If the file does not exist or is empty, header columns are written first. Data rows are appended using fputcsv($handle, $data, ',', '"', '\\') with explicit delimiter, enclosure, and escape parameters to prevent CSV injection and formatting anomalies, followed by fclose().

4. Are success and error responses displayed clearly?

Yes. When submission succeeds, an attractive Bootstrap alert-success container is displayed with confirmation details. If any field fails validation, an alert-danger container lists the specific errors with a link allowing the user to return and correct their input.

B. Implementation Details

The contact support module in pages/contact.html was connected to the server-side processor php/contact.php. Submissions undergo server validation and are safely stored in storage/contacts.csv. Similarly, student registrations are logged into storage/registrations.csv.

C. Evidence of Work

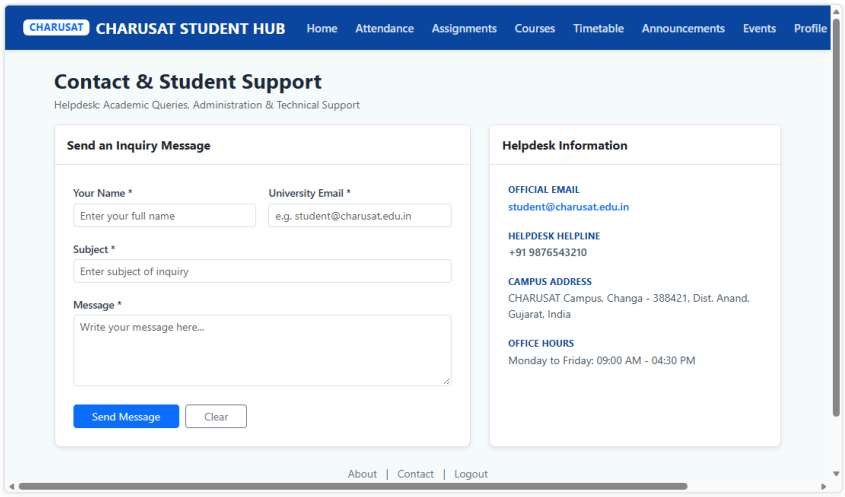


Figure: Contact Support Form with Name, Email, Subject, and Message Fields

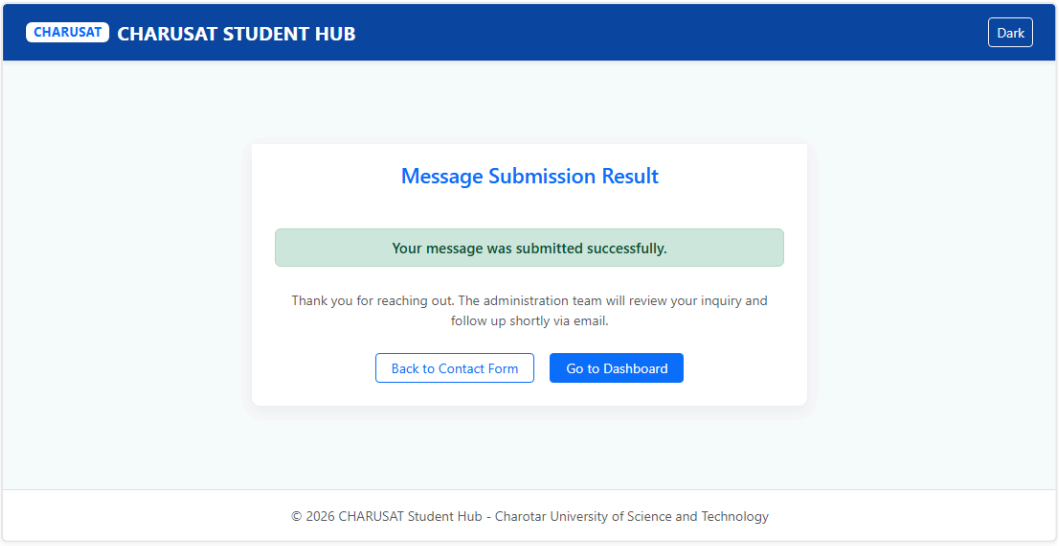


Figure: Server-Side PHP Success Response Confirming CSV Record Storage

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to process HTTP POST form data in PHP, implement server-side validation and sanitization against XSS, and safely append structured records to persistent CSV files.

Faculty Remarks: (Keep this blank for faculty)

Practical - 8

Practical Title: MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using MySQL statements, test PDO connectivity, and demonstrate prepared statements.

A. Theory & Analysis

1. Is the schema normalized properly?

Yes. The database schema studenthub_db is normalized up to Third Normal Form (3NF). Entity concerns are divided into distinct tables: authentication credentials in 'users', academic records in 'students', event details in 'events', student-event participation in 'registrations', and system audit logs in 'audit_logs'. Redundant transitive dependencies are eliminated.

2. Are primary keys and foreign keys correctly defined?

Yes. Every table defines an auto-incrementing integer primary key (id). Relational integrity is enforced using foreign keys: students.user_id references users.id; events.created_by references users.id; registrations.student_id references students.id; and registrations.event_id references events.id. A composite UNIQUE key (student_id, event_id) guarantees a student cannot register for the same event multiple times.

3. Is the database connection handled securely?

Yes. The connection helper php/db_pdo.php establishes PDO connections configured with PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, UTF-8 character encoding, and PDO::ATTR_EMULATE_PREPARES => false. Parameterized prepared statements bind query parameters safely, preventing SQL injection.

B. Implementation Details

A complete relational database studenthub_db was architected in sql/schema.sql and imported into MySQL. It contains 5 core tables (users, students, events, registrations, audit_logs) pre-seeded with 21 student records, 8 events, 7 registrations, and audit logs. A diagnostic test script in php/test_pdo.php verifies PDO connectivity and query execution.

C. Evidence of Work

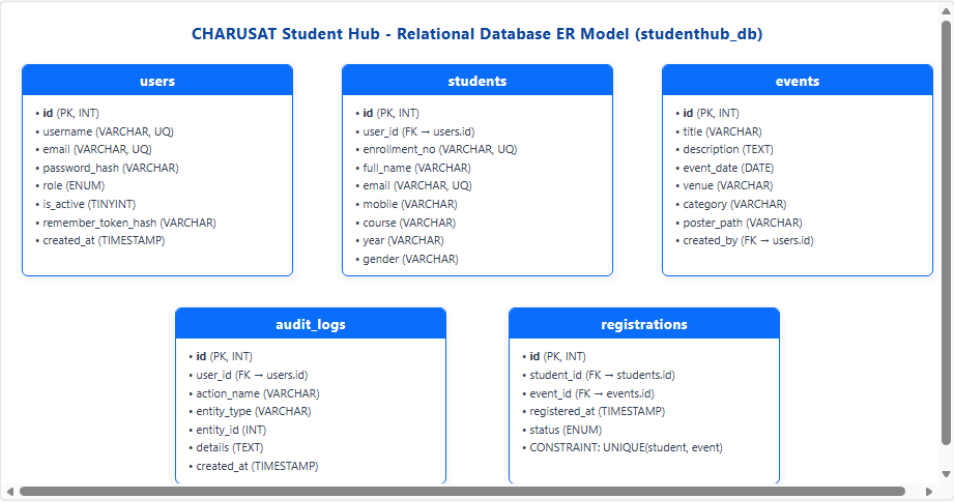


Figure: CHARUSAT Student Hub Relational Database ER Model Diagram

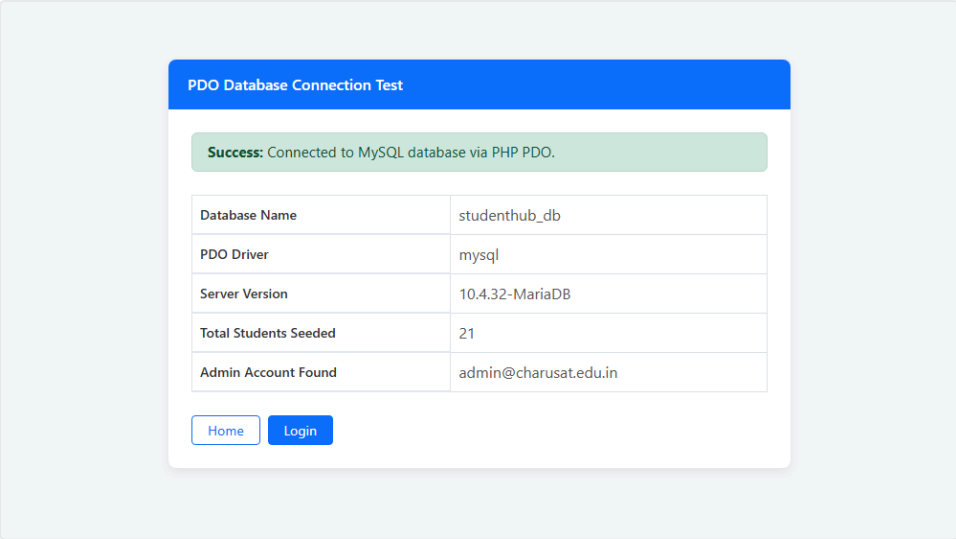


Figure: PHP PDO Database Connectivity Test and Table Diagnostic Output

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned relational database design, 3NF normalization, foreign key constraint definition, and establishing secure database connectivity using PHP PDO with prepared statements.

Faculty Remarks: (Keep this blank for faculty)

Practical - 9

Practical Title: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MySQLi. Validate input on frontend and backend, check for duplicate emails/usernames, hash passwords with password_hash(), and use prepared statements.

A. Theory & Analysis

1. Is password_hash() used before storing passwords?

Yes. Plaintext passwords submitted by users are never stored in the database. The registration handler processes passwords with password_hash($password, PASSWORD_DEFAULT), generating a cryptographically secure bcrypt hash containing an automatically generated salt and appropriate cost factor.

2. Is duplicate email/username checked?

Yes. Before attempting insertion, the script executes a prepared SELECT statement querying users for matching email or username values. If a match is detected, the transaction halts gracefully and returns an informative error banner without crashing or exposing raw database constraint exceptions.

3. Are frontend and backend validations consistent?

Yes. Consistent validation rules are enforced across both tiers: names must match letters only (2-50 characters), email must be valid format, mobile must be 10 digits starting with 6-9, and passwords must have at least 6 characters including letters and digits. Backend validation ensures security even if client-side validation is bypassed.

B. Implementation Details

The registration processor in php/register.php connects to MySQL via MySQLi. It validates input parameters, detects duplicates, hashes passwords, and executes an atomic database transaction (mysqli_begin_transaction) inserting related records into both users and students tables, while recording an audit log entry.

C. Evidence of Work

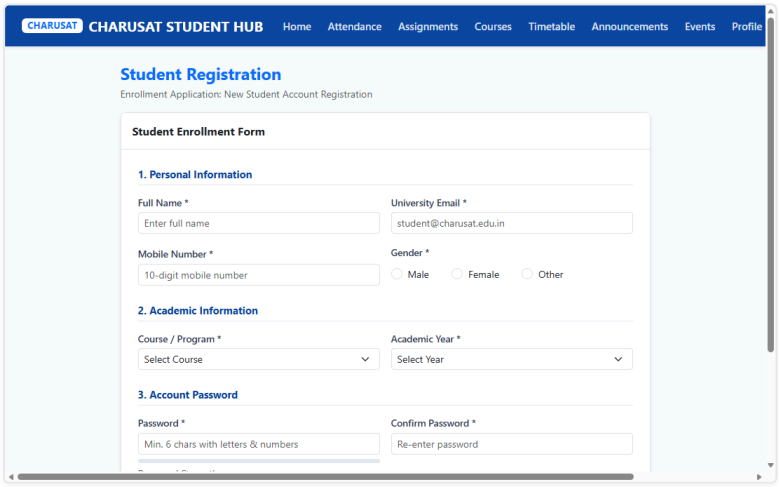


Figure: Student Registration Form with Multi-Field Validation Controls

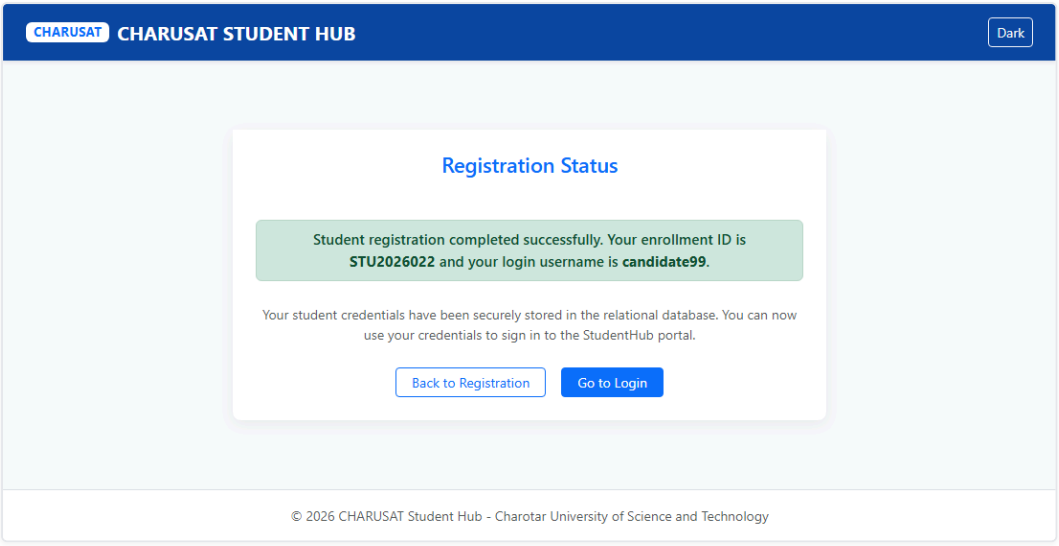


Figure: Successful Database Registration Confirmation with Allocated Student ID

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to implement secure user registration in PHP and MySQL using cryptographic password hashing, duplicate record checking, multi-table transactions, and dual-layer validation.

Faculty Remarks: (Keep this blank for faculty)

Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and secure remember-me token.

A. Theory & Analysis

1. Are sessions started and destroyed securely?

Yes. Sessions are initialized via start_session_safe() with session.cookie_httponly enabled to block cross-site scripting cookie theft. Upon login, session_regenerate_id(true) issues a new session ID to thwart session fixation. On logout, $_SESSION is cleared, the session cookie is expired in the past, and session_destroy() terminates server-side session data.

2. Is password_verify() used correctly?

Yes. The login processor php/login_process.php queries the user by username or email and validates the entered password using password_verify($password, $user['password_hash']). This function performs timing-attack safe comparisons against the stored bcrypt hash.

3. Is unauthorized dashboard access blocked?

Yes. The middleware check_auth() in php/auth.php inspects $_SESSION['user_id']. If no active session exists, the request is immediately terminated and redirected to login.html?msg=unauthorized. Role-based checks prevent students from opening admin endpoints and vice versa.

4. Is session timeout implemented?

Yes. An inactivity timeout of 30 minutes (1800 seconds) is enforced by comparing time() against $_SESSION['last_activity']. If the threshold is exceeded, the session is purged and the user is redirected to login.html?msg=timeout.

5. Are admin and student users redirected to appropriate dashboards?

Yes. Upon successful authentication, administrators are directed to pages/admin_dashboard.php while students are directed to pages/dashboard.php. In addition, persistent remember-me tokens are hashed with SHA-256 and stored in the database for secure multi-day authentication.

B. Implementation Details

Authentication was implemented in php/auth.php, php/login_process.php, and php/logout.php. The login page (pages/login.html) includes credentials verification, session ID regeneration, 30-minute inactivity timeout, role redirection, and remember-me tokens hashed in users.remember_token_hash with 14-day expiration.

C. Evidence of Work

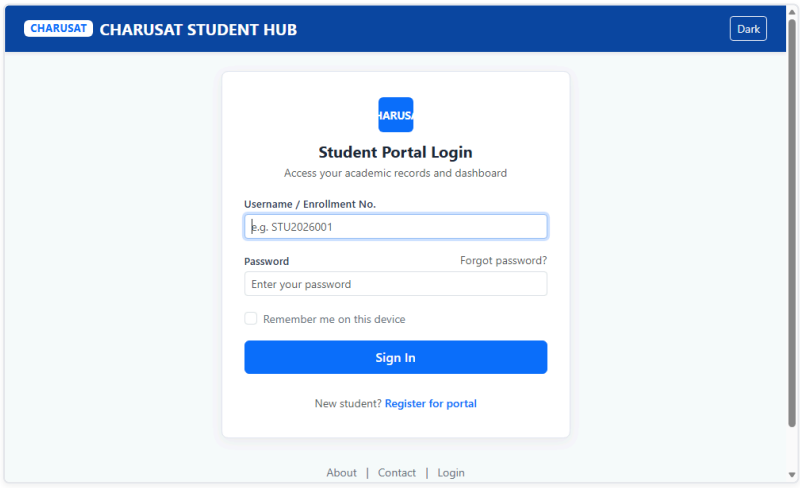


Figure: Secure Login Interface with Remember-Me and Validation Feedback

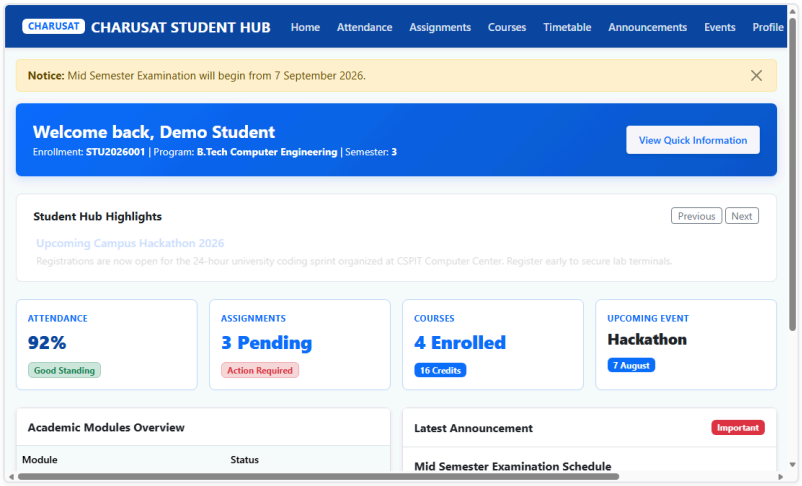


Figure: Authenticated Student Dashboard with Verified Session Identity

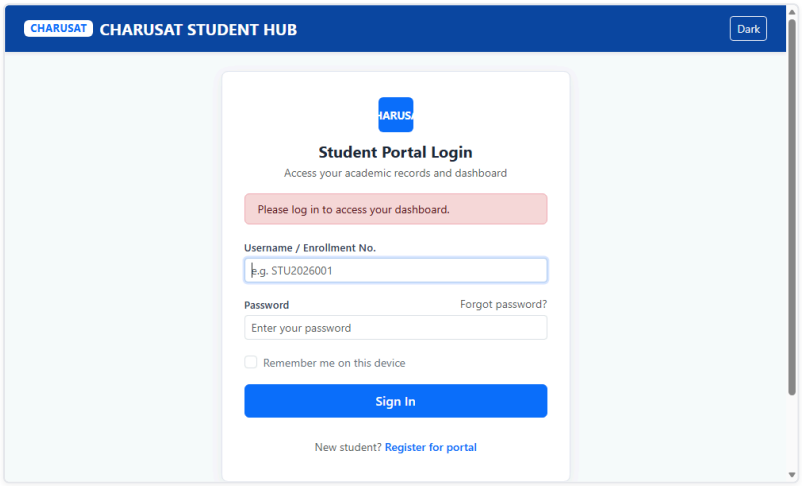


Figure: Unauthorized Route Access Prevention with Informative Alert Banner

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned to build secure session authentication in PHP, including password verification, session fixation prevention, role-based access control (RBAC), automatic timeouts, and hashed remember-me tokens.

Faculty Remarks: (Keep this blank for faculty)

Practical - 11

Practical Title: Student Management CRUD with Search and Filter

Problem Definition: Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MySQLi statements, prepared statements, and clear success/failure messages.

A. Theory & Analysis

1. Are SELECT, INSERT, UPDATE, and DELETE operations working?

Yes. All four CRUD operations are implemented using MySQLi prepared statements: SELECT queries retrieve student records with dynamic filtering; INSERT statements enroll new students after duplicate validation; UPDATE statements modify details; and DELETE statements remove records cleanly.

2. Is search by name/email/course implemented?

Yes. The query dynamically binds search keywords with SQL LIKE '%...%' across enrollment_no, full_name, and email fields. It simultaneously applies exact course filtering (e.g., 'B.Tech Computer Engineering', 'BCA') from a dropdown selector, allowing combined search and filter criteria.

3. Are delete operations confirmed before execution?

Yes. When an administrator clicks the Delete button, a confirmation modal dialog opens displaying the student's name and enrollment number. Deletion executes only upon explicit user confirmation, preventing accidental data loss.

B. Implementation Details

The administrative student management module in pages/manage_students.php provides full CRUD operations with 21 seeded student records. It includes live search, course filters, Add Student modal, Edit Student modal, Delete confirmation modal, CSV data export (?export=csv), and audit log integration.

C. Evidence of Work

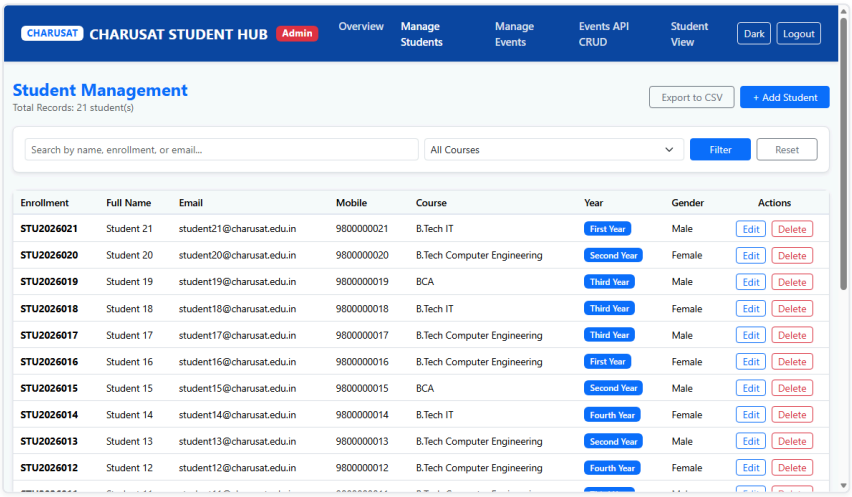


Figure: Student Management Administrative Table with Search and Action Modals

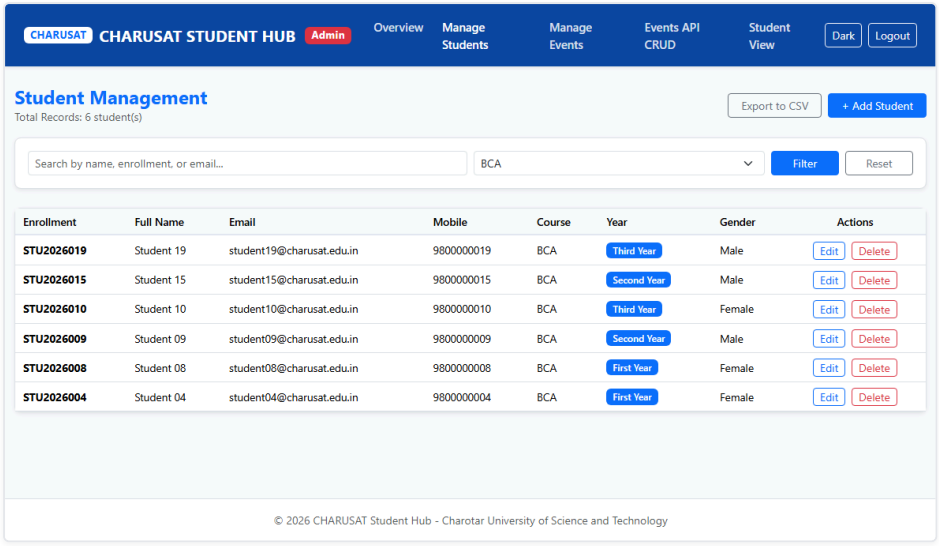


Figure: Filtered Student Records by Academic Course with Real-Time Counter

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to construct an administrative database CRUD module in PHP and MySQL using prepared statements, multi-field search, course filtering, confirmation modals, and CSV data export.

Faculty Remarks: (Keep this blank for faculty)

Practical - 12

Practical Title: Event Management CRUD with Poster Upload and File Validation

Problem Definition: Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters. Validate uploaded files on the server for allowed MIME types and maximum file size (≤ 2MB).

A. Theory & Analysis

1. Are add, update, and delete functionalities implemented correctly?

Yes. Administrators can create events with title, description, date, venue, and category; edit existing event details; and delete obsolete events. Associated poster files are automatically unlinked from the server's filesystem upon deletion.

2. Is the UI linked with the database correctly?

Yes. All event actions synchronize directly with the 'events' table in studenthub_db. The interface displays categorized event badges (upcoming, completed, cancelled) and previews uploaded event poster graphics.

3. Are poster uploads validated for type and size?

Yes. Robust server validation checks: (1) upload error codes; (2) file size limit of 2MB (2,097,152 bytes); (3) verified MIME type detection via PHP finfo_file(FILEINFO_MIME_TYPE) allowing image/jpeg, image/png, and image/webp; and (4) cryptographically random filename generation preventing executable upload vulnerabilities.

4. Are success and failure messages displayed?

Yes. Contextual Bootstrap alerts communicate validation outcomes, such as 'Event created successfully' or 'File size exceeds maximum limit of 2MB'.

B. Implementation Details

The event management module in pages/manage_events.php supports full event CRUD and secure poster uploads to uploads/posters/. Uploads are validated using PHP's Fileinfo extension for MIME types (JPEG, PNG, WebP) and 2MB file limits, with randomized safe filenames stored in MySQL.

C. Evidence of Work

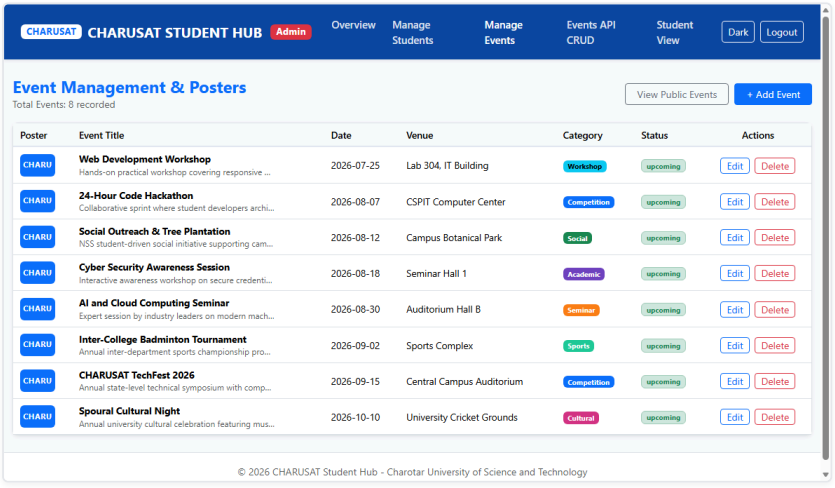


Figure: Admin Event Management Module with File Upload and Database Operations

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to implement secure file uploads in PHP with MIME validation, file size enforcement, unique sanitizing renames, and complete CRUD management in MySQL.

Faculty Remarks: (Keep this blank for faculty)

Practical - 13

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload. Show loading and error states.

A. Theory & Analysis

1. Do PHP endpoints return valid JSON responses?

Yes. The backend script api/events.php outputs HTTP header 'Content-Type: application/json; charset=utf-8' and encodes all responses with json_encode(['success' => true, ...]). Appropriate HTTP status codes (200, 400, 403, 405, 500) are returned.

2. Are GET and POST requests handled properly?

Yes. GET requests fetch, search, and filter event records. POST requests handle mutations (create, update, delete) parsed from JSON payloads or form submissions, with admin authorization enforced via session checks.

3. Are loading, success, and error states shown on the frontend?

Yes. The frontend client in pages/events_api.php displays a spinning loading indicator during network requests, disables submit buttons to prevent double submissions, and renders dynamic alert banners on success or error without refreshing the browser.

4. Is input validation performed before database operations?

Yes. Incoming request parameters are validated for non-empty fields, valid date formatting, and recognized categories before executing parameterized SQL statements.

B. Implementation Details

A REST-style PHP JSON API was developed in api/events.php supporting GET (search/filter/sort) and POST (create/update/delete) with audit logging. A responsive single-page application interface in pages/events_api.php consumes this API asynchronously via JavaScript fetch(), allowing seamless event CRUD without page reload.

C. Evidence of Work

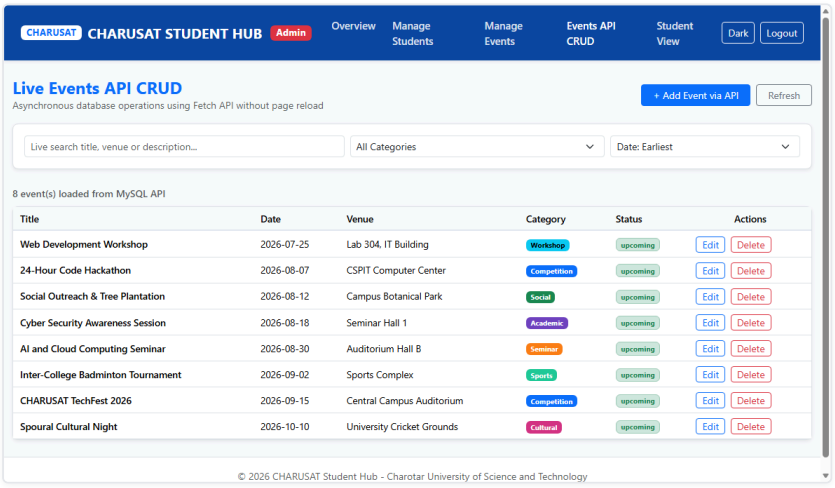


Figure: Asynchronous Event CRUD Interface Consuming PHP JSON API via Fetch API

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to build RESTful JSON API endpoints in PHP and consume them using the JavaScript Fetch API to deliver real-time, asynchronous CRUD operations without page reloads.

Faculty Remarks: (Keep this blank for faculty)

Practical - 14

Practical Title: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem Definition: Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role. Record important admin actions in audit_logs.

A. Theory & Analysis

1. Are dashboard values loaded dynamically from the database?

Yes. Key performance indicators are computed in real time using SQL aggregation queries: SELECT COUNT(*) FROM students, SELECT COUNT(*) FROM events, SELECT COUNT(*) FROM registrations, and SELECT COUNT(*) FROM users. These metrics update dynamically as records are added or removed.

2. Is access restricted to admin users?

Yes. Server-side middleware check_auth('admin') strictly enforces role checks before rendering the page. Any unauthenticated user is redirected to login.html, and authenticated students attempting to access admin files are redirected to dashboard.php with a forbidden alert.

3. Are role-based menus working?

Yes. The navigation bar dynamically adapts to the authenticated role: administrators receive management links (Overview, Manage Students, Manage Events, Events API CRUD), while students receive student-specific academic links.

4. Are important admin actions recorded in audit_logs?

Yes. The audit_logs table records security and operational actions (USER_LOGIN, CREATE_STUDENT, UPDATE_STUDENT, DELETE_STUDENT, CREATE_EVENT, UPDATE_EVENT, DELETE_EVENT) with user ID, action name, entity type, details, and timestamps, displayed in an activity log table on the admin dashboard.

B. Implementation Details

The administrative dashboard in pages/admin_dashboard.php displays live metric KPI cards, recent student enrollments, module quick links, and a dedicated System Audit Trail & Activity Log table reading directly from the audit_logs table. Strict role-based authorization prevents student access.

C. Evidence of Work

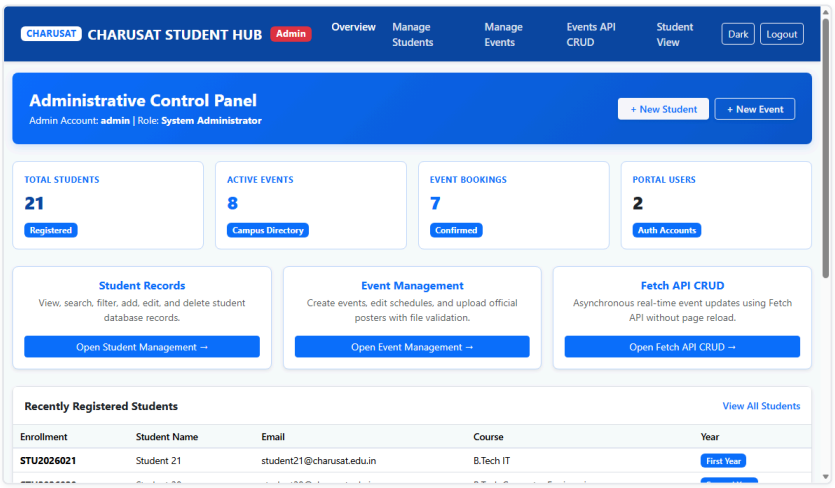


Figure: Admin Dashboard with Dynamic Metrics, Recent Enrollments, and System Audit Trail

D. Repository Link

GitHub Repository Link: https://github.com/Mr-Het-Patel/WDF

E. Learning Outcomes

I learned how to develop a secure administrative dashboard featuring dynamic SQL KPI calculations, server-enforced role-based access control, contextual navigation menus, and comprehensive audit trail logging.

Faculty Remarks: (Keep this blank for faculty)

Faculty Remarks: (Keep this blank for faculty)